

LeetCode Math Problems

1	LC-7 – Reverse Integer
2	LC-258 – Add digits
3	LC-9 – Palindrome Number
4	LC-866 – Prime Palindrome
5	LC-507 – Perfect Number
6	LC-1134 – ArmStrong Number
7	LC-326 – Power of 3
8	LC-342 – Power of 4
9	LC-50 – Power(x,n)



LeetCode7.java

//7. Reverse Integer

```
public class LeetCode7 {  
  
    public static int reverse(int num) {  
        long rev=0;  
        while(num!=0) {  
            rev = rev*10+num%10;  
            num=num/10;  
        }  
  
        if(rev<Integer.MIN_VALUE || rev>Integer.MAX_VALUE)  
            rev=0;  
  
        return (int)rev;  
    }  
  
    public static void main(String[] args) {  
        int num=1534236469;  
        int result = reverse(num);  
        System.out.println(result);  
    }  
}
```



LeetCode258.java

//258. Add Digits

```
public class LeetCode258 {  
  
    public static int addDigits(int num) {  
        int sum = 10;  
        while (sum >= 10) {  
            sum = 0;  
            while (num != 0) {  
                sum = sum + num % 10;  
                num = num / 10;  
            }  
            System.out.println(sum);  
            num = sum;  
        }  
        return sum;  
    }  
  
    public static void main(String[] args) {  
        int num = 123456;  
        int result = addDigits(num);  
        System.out.println("Result : "+result);  
    }  
}
```



LeetCode9.java

//9. Palindrome Number

```
public class LeetCode9 {  
  
    public static boolean isPalindrome(int num) {  
        if (num < 0)  
            return false;  
  
        int ncopy = num;  
  
        int rev = 0;  
        while (num != 0) {  
            rev = rev * 10 + num % 10;  
            num = num / 10;  
        }  
  
        return (ncopy == rev);  
    }  
  
    public static void main(String[] args) {  
        int num = 1232;  
        boolean result = isPalindrome(num);  
        System.out.println(result);  
    }  
}
```



LeetCode866.java

//866. Prime Palindrome

```
public class LeetCode866 {  
  
    public static boolean isPalindrome(int num) {  
        if (num < 0)  
            return false;  
  
        int ncopy = num;  
  
        int rev = 0;  
        while (num != 0) {  
            rev = rev * 10 + num % 10;  
            num = num / 10;  
        }  
  
        return (ncopy == rev);  
    }  
  
    public static boolean isPrime(int num) {  
  
        if (num < 2)  
            return false;  
  
        for (int i = 2; i <= Math.sqrt(num); i++) {  
            if (num % i == 0)  
                return false;  
        }  
  
        return true;  
    }  
}
```



```
public static int primePalindrome(int num) {  
  
    if (isPrime(num) && isPalindrome(num))  
        return num;  
  
    while (true) {  
        if (isPrime(num) && isPalindrome(num))  
            return num;  
        num++;  
        if (10_000_000 < num && num < 100_000_000)  
            num = 100_000_000;  
    }  
}  
  
public static void main(String[] args) {  
    int num = 13;  
    int result = primePalindrome(num);  
    System.out.println(result);  
}  
}
```



LeetCode507.java

//507. Perfect Number

```
public class LeetCode507 {  
    public static boolean checkPerfectNumber(int num) {  
  
        if (num % 2 != 0) {  
            return false;  
        }  
  
        int sum = 0;  
        for (int i = 1; i <= num / 2; i++) {  
            if (num % i == 0)  
                sum += i;  
        }  
  
        return (sum == num);  
    }  
  
    public static void main(String[] args) {  
        int num = 28;  
        boolean result = checkPerfectNumber(num);  
        System.out.println(result);  
    }  
}
```



LeetCode1134.java

//1134 - Armstrong Number

```
public class LeetCode1134 {  
  
    public static boolean isArmstrong(int num) {  
        int k = (num + "").length();  
        int sum = 0;  
        int numCopy=num;  
        while(num>0) {  
            int digit = num%10;  
            sum = sum + (int)Math.pow(digit, k);  
            num = num/10;  
        }  
  
        return (sum == numCopy);  
    }  
  
    public static void main(String[] args) {  
        boolean result = isArmstrong(153);  
        System.out.println(result);  
    }  
}
```



LeetCode326.java

//326. Power of Three

```
public class LeetCode326 {  
    public static boolean isPowerOfThree(int num) {  
        if (num == 0) return false;  
  
        while (num % 3 == 0) {  
            num = num / 3;  
        }  
        return (num == 1);  
    }  
}
```

LeetCode326R.java

//326. Power of Three

```
public class LeetCode326R {  
    public static boolean isPowerOfThree(int num) {  
        if (num == 0)  
            return false;  
  
        if (num == 1)  
            return true;  
  
        if (num % 3 != 0)  
            return false;  
  
        return isPowerOfThree(num / 3);  
    }  
}
```



LeetCode342.java

//342. Power of Four

```
public class LeetCode342 {  
    public static boolean isPowerOfFour(int num) {  
        if (num == 0) {  
            return false;  
        }  
        while (num % 4 == 0) {  
            num = num / 4;  
        }  
        return (num == 1);  
    }  
}
```

LeetCode342R.java

//342. Power of Four

```
public class LeetCode342R{  
    public static boolean isPowerOfFour(int num) {  
  
        if (num == 0) return false;  
  
        if (num == 1) return true;  
  
        if (num % 4 != 0)  
            return false;  
  
        return isPowerOfFour(num / 4);  
    }  
}
```



LeetCode50.java

//50. Pow(x, n)

```
public class LeetCode50 {  
    public static double power(double x, long n) {  
  
        if (n == 0)  
            return 1;  
  
        if (n == 1)  
            return x;  
  
        if (n < 0)  
            return power(1 / x, -n);  
  
        double result = power(x * x, n / 2);  
  
        if (n % 2 == 1)  
            result = result * x;  
  
        return result;  
    }  
  
    public static double myPow(double x, int n) {  
        return power(x, n);  
    }  
}
```